Programming Tasks

Functions Returning Functions, (Closures)

Problem 1: Smart Calculator Factory

Scenario

A software company is building a **Smart Calculator** application. Instead of writing separate functions for every mathematical operation, they want a factory function that creates the required operation dynamically.

Problem Statement

Create a function named **`operationFactory(operation)`** that accepts a string representing a mathematical operation.

The function should return another function that accepts **two numbers** and performs the requested operation.

Supported operations are:

- `"add"`
- `"subtract"`
- `"multiply"`
- `"divide"`

If an invalid operation is provided, the returned function should display an appropriate error message.

Example

```
const add = operationFactory("add");

console.log(add(10, 20)); // 30

const multiply = operationFactory("multiply");

console.log(multiply(5, 8)); // 40
```

Problem 2: Personalized Greeting Generator

Scenario

A company is developing an employee management system. Every employee should receive a personalized welcome message whenever they log into the system. Since each employee has a different name, the system should generate a customized greeting function for every employee.

Problem Statement

Create a function named **`makeGreeter(name)`**.

The function should return another function that prints a personalized greeting message using the provided name.

Example

```
const greetAli = makeGreeter("Ali");

greetAli();
```

Output

```
Hello, Ali! Welcome to the system.
```

Problem 3: Power Function Builder

Scenario

A scientific calculator application frequently performs exponent calculations. To avoid repeatedly specifying the exponent, the developers want to create reusable power functions for squares, cubes, fourth powers, and so on.

Problem Statement

Create a function named **`powerFunction(exponent)`**.

The function should return another function that accepts a number and raises it to the specified exponent.

Example

```
const square = powerFunction(2);

console.log(square(5)); // 25
```

```
const cube = powerFunction(3);

console.log(cube(4)); // 64
```

Problem 4: Shopping Discount Calculator

Scenario

An online shopping website offers different discount percentages to different types of customers, such as Students, Premium Members, and VIP Customers. Instead of calculating the discount every time, the company wants to create reusable discount calculators.

Problem Statement

Create a function named **`discountMaker(discountPercentage)`**.

The function should return another function that accepts the original product price and returns the final price after applying the discount.

Example

```
const studentDiscount = discountMaker(15);

console.log(studentDiscount(2000));
```

Output

```
1700
```

Problem 5: Employee Salary Bonus Calculator

Scenario

A company gives different bonus percentages to employees based on their department. Instead of calculating bonuses manually every time, the HR department wants to generate bonus calculators for each department.

Problem Statement

Create a function named **`bonusCalculator(bonusPercentage)`.**

The function should return another function that accepts an employee's basic salary and returns the final salary after adding the bonus.

Example

```
const managerBonus = bonusCalculator(20);

console.log(managerBonus(50000));
```

Output

```
60000
```

Another example:

```
const developerBonus = bonusCalculator(10);

console.log(developerBonus(80000));
```

Output

```
88000
```

Programming Tasks

User Defined Objects, Arrays and Functions

Problem 1: Restaurant Order Management

Scenario

A restaurant receives multiple customer orders throughout the day.

Problem Statement

Each order contains:

- Order ID
- Customer Name
- Food Item
- Quantity
- Price
- Delivered (true/false)

Your program should:

1. Display all orders.
2. Calculate total sales.
3. Display pending orders.
4. Find the most expensive order.
5. Count delivered orders.
6. Add a new order.
7. Mark an order as delivered.
8. Display customer-wise bills.

Problem 2: Hotel Reservation System

Scenario

A hotel keeps records of all room reservations.

Problem Statement

Each reservation contains:

- Reservation ID
- Guest Name
- Room Number
- Room Type
- Number of Days
- Checked In

Your program should:

1. Display all reservations.
2. Calculate total income.
3. Display available rooms.
4. Display guests currently checked in.
5. Search by room number.
6. Add a reservation.
7. Check out a guest.
8. Print hotel occupancy statistics.

Problem 3: Vehicle Rental System

Scenario

A car rental company wants to manage its rental vehicles.

Problem Statement

Each vehicle contains:

- Vehicle ID
- Model
- Brand
- Daily Rent
- Available

Your program should:

1. Display all vehicles.
2. Display available vehicles.
3. Count rented vehicles.
4. Search by model.
5. Add a vehicle.
6. Rent or return a vehicle.
7. Find the most expensive vehicle.
8. Display total fleet information.

Problem 4: Movie Ticket Booking System

Scenario

A cinema wants to manage movie ticket bookings.

Problem Statement

Each booking contains:

- Booking ID
- Customer Name
- Movie Name
- Number of Tickets
- Ticket Price
- Payment Status

Your program should:

1. Display all bookings.
2. Calculate total revenue.
3. Display unpaid bookings.
4. Count tickets sold for each movie.
5. Find the customer who purchased the most tickets.
6. Add a booking.
7. Update payment status.
8. Print booking statistics.

Programming Tasks

Objects, Arrays Destructuring

Problem 1: Employee Profile Viewer

Scenario

A company's HR department wants to display an employee's basic information without repeatedly accessing each object property.

Problem Statement

Create an employee object containing the following information:

- Employee ID
- Name
- Department
- Salary
- Email
- City

Write a function `displayEmployee(employee)` that uses **object destructuring** to extract the employee's **name, department, and salary**, then display them in a readable format.

Problem 2: Student Result Summary

Scenario

A school stores students' marks in an array. The principal only wants to display marks of the first three subjects separately.

Problem Statement

Create an array containing marks of six subjects.

Write a function `printMarks(marks)` that uses **array destructuring** to extract the marks of the **first three subjects** and display them.

Ignore the remaining subjects.

Problem 3: Online Shopping Receipt

Scenario

An e-commerce website stores product details in objects. During checkout, only selected information is required to print the customer's receipt.

Problem Statement

Create a product object containing:

- Product ID
- Name
- Price
- Brand
- Category
- Stock

Use **object destructuring** to extract:

- Product Name
- Price
- Brand

Display the receipt in a user-friendly format.

Problem 4: Hospital Patient Card

Scenario

A hospital management system stores complete patient records, but doctors only need essential information during consultation.

Problem Statement

Create a patient object containing:

- Patient ID
- Name
- Age
- Disease
- Blood Group

- Doctor Name

Write a function that uses **object destructuring** to extract only:

- Name
- Age
- Disease

Display the patient's consultation card.

Problem 5: University Course Information

Scenario

A university stores information about each course in an object. Students only need to view a few important details while selecting courses.

Problem Statement

Create a course object containing:

- Course Code
- Course Name
- Credit Hours
- Instructor
- Semester
- Department

Use **object destructuring** to extract:

- Course Name
- Instructor
- Credit Hours

Display the extracted information.

Programming Tasks

Rest Operator Problems

Problem 1: Student Course Registration

Scenario

A university allows students to register for any number of elective courses.

Problem Statement

Create a function `registerCourses(studentName, ...courses)`.

The function should:

- Accept the student's name.
- Accept any number of course names using the **rest operator**.
- Display the student's name followed by the list of registered courses.

Problem 2: Restaurant Order System

Scenario

A restaurant allows customers to order multiple food items in a single order.

Problem Statement

Create a function `placeOrder(customerName, ...foodItems)`.

The function should:

- Accept the customer's name.
- Accept any number of food items.
- Display the complete order.

Problem 3: Company Attendance System

Scenario

An HR department records attendance for employees attending a training workshop.

Problem Statement

Create a function `markAttendance(trainingName, ...employees)`.

The function should:

- Accept the training session name.
- Accept any number of employee names.
- Display the training name and the list of employees who attended.

Problem 4: Online Shopping Cart

Scenario

An online shopping application allows customers to purchase multiple products in a single transaction.

Problem Statement

Create a function `checkout(customerName, ...products)`.

The function should:

- Accept the customer's name.
- Accept any number of product names.
- Display the shopping summary, including all purchased products.

Problem 5: Event Registration System

Scenario

A conference organizer allows participants to register in multiple workshops during the event.

Problem Statement

Create a function `registerWorkshops(participantName, ...workshops)`.

The function should:

- Accept the participant's name.
- Accept any number of workshop names.

- Display the participant's registration details, including all selected workshops.

Bonus Challenge (Combining Destructuring + Rest Operator)

Scenario

A company stores employee details in an object. Besides the basic details, each employee can have any number of technical skills.

Problem Statement

Create an employee object containing:

- Name
- Department
- Salary
- Skills (Array)

Write a function that:

1. Uses **object destructuring** to extract the employee's **name** and **department**.
2. Uses the **rest operator** to accept any number of additional certifications.
3. Displays the employee's basic information along with the newly provided certifications.

This exercise helps students understand how destructuring and the rest operator can work together in real-world applications.

Programming Tasks

Objects, Arrays Destructuring

Problem 1: Student Enrollment System

Scenario

A school maintains a list of enrolled students. Whenever a new student joins, the school wants to create a **new updated list** without modifying the original student list.

Problem Statement

Create an array of student objects.

Write a function `enrollStudent(students, newStudent)` that:

- Accepts the existing array of students.
- Accepts a new student object.
- Uses the **spread operator** to create a new array containing all existing students along with the newly enrolled student.
- Displays the updated student list.

Note: Do not modify the original array.

Problem 2: Employee Information Update

Scenario

A company's HR department needs to update employee information whenever an employee gets promoted or transferred. The original employee record should remain unchanged for auditing purposes.

Problem Statement

Create an employee object containing:

- Employee ID
- Name

- Department
- Salary
- City

Write a function `updateEmployee(employee, updatedDetails)` that:

- Accepts an employee object.
- Accepts another object containing updated information.
- Uses the **spread operator** to create a new employee object with the updated values.
- Displays the updated employee record.

Note: The original object must remain unchanged.

Problem 3: Online Shopping Cart Merge

Scenario

An online shopping website stores items in separate carts. When a customer logs into their account, the guest cart should be merged with their existing account cart.

Problem Statement

Create two arrays of product objects:

- Guest Cart
- User Cart

Write a function `mergeCarts(userCart, guestCart)` that:

- Uses the **spread operator** to combine both carts into a new array.
- Displays the merged shopping cart.

Note: Neither of the original carts should be modified.

Problem 4: Course Registration Update

Scenario

A university allows students to register for additional courses during the semester. The updated registration should be stored separately while preserving the original record.

Problem Statement

Create a student object containing:

- Student ID
- Name
- Department
- Registered Courses (Array)

Write a function `addCourse(student, newCourse)` that:

- Uses the **spread operator** to create a new student object.
- Adds the new course to the existing list of registered courses.
- Displays the updated registration details.

Note: The original student object should remain unchanged.

Problem 5: Restaurant Menu Expansion

Scenario

A restaurant introduces new food items every month. The management wants to publish an updated menu without changing the previous month's menu.

Problem Statement

Create an array containing the current menu items.

Write a function `updateMenu(menu, newItems)` that:

- Accepts the existing menu.
- Accepts an array of newly introduced food items.
- Uses the **spread operator** to create a new menu containing both old and new items.
- Displays the updated menu.

Note: The original menu should remain unchanged.